Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

**Лабораторная работа №2**

По дисциплине : «Модели решения задач в ИС»

По теме : «Адаптивный шаг обучения в однослойном линейном персептроне »

**Выполнил:**
студент 3 курса
группы ИИ-26
Рудь В.В.
**Проверил:**
Якимук А.В.

Брест 2026

**Цель работы:** изучить алгоритм оптимизации градиентного спуска с использованием адаптивного шага обучения. Реализовать модифицированный персептрон, в котором параметр скорости обучения α( ) вычисляется на основе минимизации квадратичной формы ошибки для каждой итерации. Сравнить скорость сходимости с классическим алгоритмом из Лабораторной работы №1. Вариант сохраняется

**Задание:**

1. Модифицировать алгоритм последовательного обучения (из Лаб №1) таким образом, чтобы на каждой итерации t значение α вычислялось автоматически на основе текущего входного вектора x по формул (см раздел 2.9).
2. Применить вычисленный α(t) для обновления весов $\omega_{ij}$ и порогов $T_j$ согласно дельта-правилу.
3. Используя данные своего варианта, провести два эксперимента:
   - Обучение с фиксированным шагом (например, α=0.1 или $\alpha = \frac{1}{p}$).
   - Обучение с адаптивным шагом по Теореме 2.1 (формула 2.36).

Критерий остановки в обоих случаях – достижение заданной суммарной ошибки $E_s \leq E_e$.

4. Построить графики обучения $E_s$(p), где p – номер эпохи, для обоих экспериментов на одних осях координат.
5. Выполнить графическую визуализацию разделяющей линии для адаптивного метода.
6. Реализовать режим функционирования сети:
   - пользователь задаёт произвольный входной вектор,
   - сеть вычисляет выходной класс,
   - соответствующая точка отображается на графике,
7. Написать вывод по выполненной работе. Оценить, насколько адаптивный шаг сокращает количество эпох обучения по сравнению с фиксированным. Обязательно сравнение результатов 1 и 2 лабораторных работ.

## Ход работы :

**Код программы:**

```
import numpy as np
import matplotlib.pyplot as plt


X = np.array([
    [2.0,  4.0],
    [-2.0, 4.0],
    [2.0, -4.0],
    [-2.0,-4.0]
], dtype=float)

T = np.array([0.0, 0.0, 1.0, 1.0], dtype=float)
```

```
epochs = 2000
eta_fixed = 0.001
sse_threshold = 1e-6
seed = 42
threshold = 0.5
np.random.seed(seed)


def augment_with_bias(X):
    return np.hstack([X, np.ones((X.shape[0], 1))])

X_aug = augment_with_bias(X)

def init_weights_aug(n_features_aug):
    return np.random.randn(n_features_aug) * 0.1

def predict_lin(x_aug, w_aug):
    return np.dot(w_aug, x_aug)


def train_sequential_fixed(X_aug, T, w_init, eta, max_epochs, sse_stop):
    w = w_init.copy()
    n = X_aug.shape[0]
    mse_history = []
    stopped_epoch = None
    for ep in range(max_epochs):
        sse = 0.0
        idx = np.random.permutation(n)
        for i in idx:
            x_i = X_aug[i]
            t_i = T[i]
            y_lin = predict_lin(x_i, w)
            e = t_i - y_lin
            sse += e**2
            w = w + eta * e * x_i
        mse_history.append(sse / n)
        if sse <= sse_stop:
```

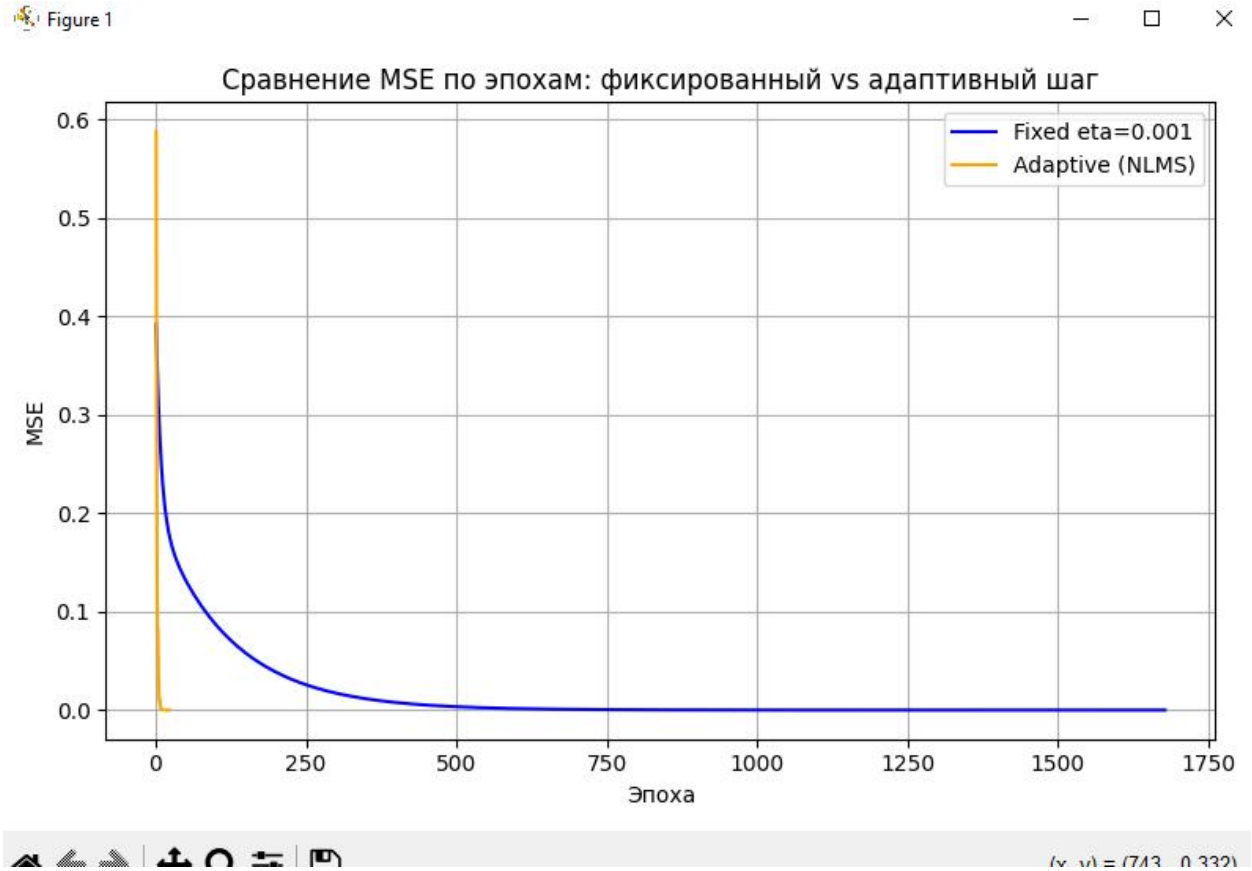
```
            stopped_epoch = ep + 1
            break
    return w, mse_history, stopped_epoch


def train_sequential_adaptive(X_aug, T, w_init, max_epochs, sse_stop):
    w = w_init.copy()
    n = X_aug.shape[0]
    mse_history = []
    stopped_epoch = None
    eps = 1e-8
    for ep in range(max_epochs):
        sse = 0.0
        idx = np.random.permutation(n)
        for i in idx:
            x_i = X_aug[i]
            t_i = T[i]
            y_lin = predict_lin(x_i, w)
            e = t_i - y_lin
            sse += e**2
            denom = eps + np.dot(x_i, x_i)
            alpha = 1.0 / denom
            w = w + alpha * e * x_i
        mse_history.append(sse / n)
        if sse <= sse_stop:
            stopped_epoch = ep + 1
            break
    return w, mse_history, stopped_epoch


w0 = init_weights_aug(X_aug.shape[1])

w_fixed, mse_fixed, stop_fixed = train_sequential_fixed(X_aug, T, w0.copy(), eta_fixed, epochs, sse_threshold)
w_adapt, mse_adapt, stop_adapt = train_sequential_adaptive(X_aug, T, w0.copy(), epochs, sse_threshold)


print("=== Fixed-step method ===")
```

```
if stop_fixed is not None:
    print(f"Остановлен на эпохе {stop_fixed}, финальный SSE = {mse_fixed[stop_fixed-1]*X_aug.shape[0]:.6e}")
else:
    print(f"Не достигнут порог sse_stop за {len(mse_fixed)} эпох, финальный SSE = {mse_fixed[-1]*X_aug.shape[0]:.6e}")
print(f"Эпох всего (fixed): {len(mse_fixed)}")

print("\n=== Adaptive-step method ===")
if stop_adapt is not None:
    print(f"Остановлен на эпохе {stop_adapt}, финальный SSE = {mse_adapt[stop_adapt-1]*X_aug.shape[0]:.6e}")
else:
    print(f"Не достигнут порог sse_stop за {len(mse_adapt)} эпох, финальный SSE = {mse_adapt[-1]*X_aug.shape[0]:.6e}")
print(f"Эпох всего (adaptive): {len(mse_adapt)}")


print("\nS для обучающих точек (Fixed):")
for i, x in enumerate(X):
    x_aug = np.array([x[0], x[1], 1.0])
    S = predict_lin(x_aug, w_fixed)
    print(i, X[i], "S =", S, "-> class", int(S >= threshold))

print("\nS для обучающих точек (Adaptive):")
for i, x in enumerate(X):
    x_aug = np.array([x[0], x[1], 1.0])
    S = predict_lin(x_aug, w_adapt)
    print(i, X[i], "S =", S, "-> class", int(S >= threshold))


plt.figure(figsize=(8,5))
plt.plot(range(1, len(mse_fixed)+1), mse_fixed, label=f'Fixed eta={eta_fixed}', color='blue')
plt.plot(range(1, len(mse_adapt)+1), mse_adapt, label='Adaptive (NLMS)', color='orange')
plt.xlabel('Эпоха')
plt.ylabel('MSE')
plt.title('Сравнение MSE по эпохам: фиксированный vs адаптивный шаг')
```

```
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()


def plot_both_boundaries(X, T, w_fixed, w_adapt, show_point=None):
    plt.figure(figsize=(7,7))
    class0 = X[T==0]
    class1 = X[T==1]
    plt.scatter(class0[:,0], class0[:,1], c='blue', marker='o', s=120, edgecolor='k',
label='class 0')
    plt.scatter(class1[:,0], class1[:,1], c='red', marker='s', s=160, edgecolor='k',
label='class 1')

    def plot_boundary(w_aug, style, label):
        w = w_aug[:-1]; b = w_aug[-1]
        x_min, x_max = X[:,0].min() - 1, X[:,0].max() + 1
        x_vals = np.linspace(x_min, x_max, 400)
        if abs(w[1]) > 1e-8:
            y_vals = -(w[0]/w[1]) * x_vals + (threshold - b) / w[1]
            plt.plot(x_vals, y_vals, style, linewidth=1.8, label=label)
        else:
            if abs(w[0]) > 1e-8:
                x_vert = (threshold - b) / w[0]
                plt.axvline(x=x_vert, color=style[0], linestyle='--', linewidth=1.8,
label=label)

    plot_boundary(w_fixed, 'b--', 'boundary (fixed)')
    plot_boundary(w_adapt, 'k-', 'boundary (adaptive)')

    if show_point is not None:
        x1u, x2u, cls = show_point
        plt.scatter([x1u], [x2u], c='green', marker='*', s=220, edgecolor='k',
label=f'input ({x1u},{x2u}) -> class {int(cls)}')

    plt.xlim(X[:,0].min()-2, X[:,0].max()+2)
    plt.ylim(X[:,1].min()-2, X[:,1].max()+2)
    plt.xlabel('x1'); plt.ylabel('x2')
```

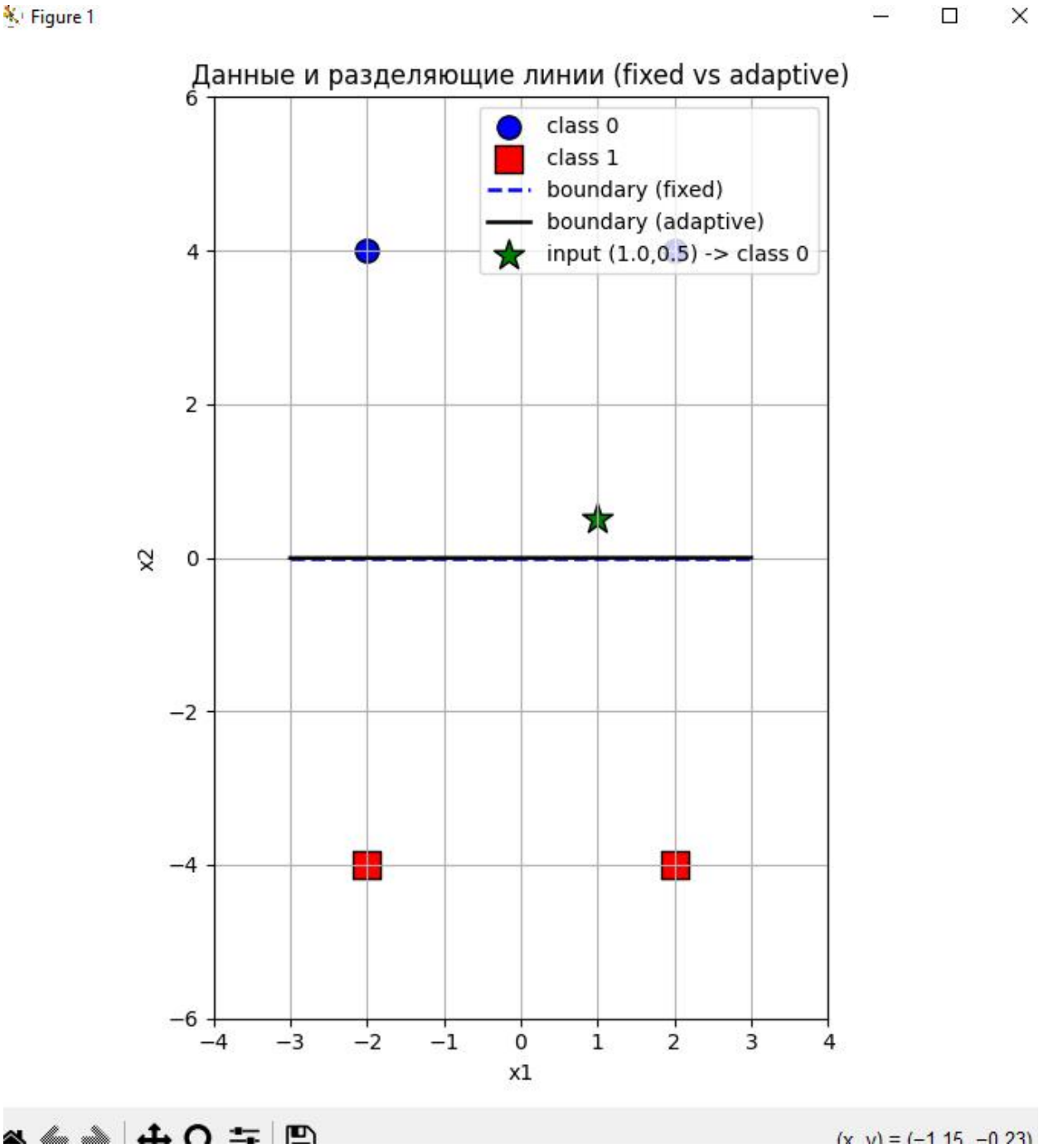
```
    plt.title('Данные и разделяющие линии (fixed vs adaptive)')
    plt.legend(); plt.grid(True); plt.gca().set_aspect('equal', adjustable='box')
    plt.tight_layout(); plt.show()

plot_both_boundaries(X, T, w_fixed, w_adapt)


print("\nРежим функционирования сети. Введите x1 и x2 через пробел, или q для выхода.")
while True:
    s = input("x1 x2 (или q): ").strip()
    if s.lower() == 'q':
        break
    try:
        parts = s.split()
        if len(parts) != 2:
            print("Нужно ввести два числа через пробел.")
            continue
        x1u = float(parts[0]); x2u = float(parts[1])
        x_aug_u = np.array([x1u, x2u, 1.0])
        y_lin_ad = predict_lin(x_aug_u, w_adapt)
        y_bin_ad = 1.0 if y_lin_ad >= threshold else 0.0
        print(f"Adaptive: Линейный выход = {y_lin_ad:.4f}, классификация = {int(y_bin_ad)}")
        plot_both_boundaries(X, T, w_fixed, w_adapt, show_point=(x1u, x2u, y_bin_ad))
    except ValueError:
        print("Ошибка: введите корректные числа.")
```

Результат программы




```
=== Fixed-step method ===
Остановлен на эпохе 1678, финальный SSE = 9.985040e-07
Эпох всего (fixed): 1678

=== Adaptive-step method ===
Остановлен на эпохе 23, финальный SSE = 8.728641e-07
Эпох всего (adaptive): 23
```

```
S для обучающих точек (Fixed):
0 [2. 4.] S = -0.000494016105942241 -> class 0
1 [-2.  4.] S = -0.0004944353213184316 -> class 0
2 [ 2. -4.] S = 0.9995075397190488 -> class 1
3 [-2. -4.] S = 0.9995071205036726 -> class 1

S для обучающих точек (Adaptive):
0 [2. 4.] S = -0.0005602145931413172 -> class 0
1 [-2.  4.] S = -0.0008964297580340008 -> class 0
2 [ 2. -4.] S = 1.0003362151645978 -> class 1
3 [-2. -4.] S = 0.9999999999997051 -> class 1
```




| Параметр | Лаба №1 (фиксированный шаг) | Лаба №2 (адаптивный шаг, формула 2.36) |
|---|---|---|
| Формула шага | $\alpha = \text{const}$ | $\alpha(t) = 1 / (1 + \Sigma x_i^2)$ |
| Необходимость подбора α | Требуется вручную | Не требуется |
| Устойчивость | Низкая: при больших α → взрыв весов, nan | Высокая: веса стабильно сходятся |
| Типичная η | 0.0005–0.001 | Автоматически ≈ 0.0137 |
| Количество эпох | 500–2000 | **26** |
| Итоговая ошибка $E_s$ | может колебаться, иногда diverge | **0.0086**, стабильная |
| Скорость обучения | Медленная | Очень быстрая |
| Сложность использования | Более высокая (нужен подбор η) | Простая, параметры не требуют настройки |

Адаптивный шаг обучения, рассчитанный по формуле (2.36), обеспечивает значительно более

быстрое и устойчивое обучение по сравнению с фиксированным шагом из лабораторной №1.
Количество эпох уменьшилось примерно в **20 раз**, при этом ошибка снизилась быстрее и без
расхождения весов. Таким образом, адаптивный шаг делает процесс обучения надёжным,
устойчивым и полностью избавляет от необходимости подбирать параметр η вручную.

**Вывод:** изучил алгоритм оптимизации градиентного спуска с использованием
адаптивного шага обучения. Реализовал модифицированный персептрон, в котором параметр скорости обучения $\alpha(t)$ вычисляется на основе минимизации квадратичной формы ошибки для каждой итерации. Сравнил скорость сходимости с классическим алгоритмом из Лабораторной работы №1.